

AI Calling Agent Cost Reduction Plan

How we cut cost per call from ₹12 → ₹3 without hurting quality

Prepared for: Engineering & Founders — team review

Scope: Unit economics (cost per call) + platform overhead

Version: 1.0 · **Status:** Draft for discussion

One-line summary. Our biggest cost lever (dropping LiveKit for the standard tier) is **already built** in the codebase. Combined with TTS caching, prompt trimming, a model cascade, and self-hosting speech at scale, we can cut delivery cost by **~75%** — saving an estimated **₹27 lakh/month at 1,000 customers** — while keeping a premium high-quality tier intact.

1. Executive Summary

Our product has real cost-of-goods (voice AI is not a zero-marginal-cost business). Today each 3-minute call costs about **₹12 (\$0.125)** to deliver. Most of that is Text-to-Speech and the LiveKit real-time layer. We can bring it down in two stages.

Today (LiveKit stack)

₹12 / call

≈ ₹4.0 / minute

Stage 1 — quick wins

₹5.6 / call

no new infrastructure

Stage 2 — at scale

₹3.0 / call

self-hosting + caching

What this means in money

Customers (tenants)	Save/mo — Stage 1 (₹6.4/call)	Save/mo — Stage 2 (₹9/call)
100	₹1.9 L	₹2.7 L
500	₹9.6 L	₹13.5 L
1,000	₹19.2 L	₹27.0 L
5,000	₹96 L	₹1.35 Cr
10,000	₹1.92 Cr	₹2.70 Cr

Assumes ~300 answered calls/tenant/month. Formula: $\text{saving} = \text{users} \times 300 \times (\text{₹12} - \text{target})$.

2. Where the money goes (per call)

Two components — **TTS (40%)** and **LiveKit (24%)** — are two-thirds of the cost. That's exactly where we focus.

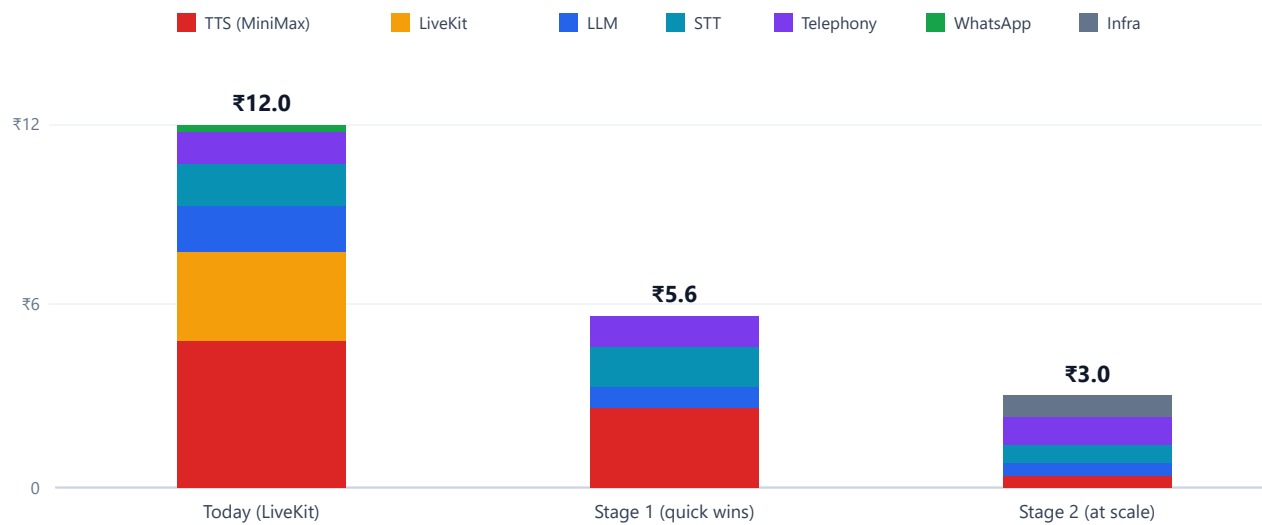


Figure 1 — Cost per 3-minute call, by component, across the three stages.

Component	Today	Why	Action
TTS — MiniMax	₹4.8	Re-synthesized every reply	Cache static lines → self-host at scale
LiveKit Cloud	₹2.9	\$0.01/min real-time layer	Use built-in WebSocket path for standard tier
LLM — Groq	₹1.5	1,800-token prompt re-sent each turn	Trim prompt + model cascade + cache
STT — Deepgram	₹1.4	Per-minute streaming	Self-host (faster-whisper) at scale
Telephony — Vobiz	₹1.05	Per-minute inbound	Volume commit
WhatsApp	₹0.2	Template messages	Send inside free 24h window

3. Current architecture (annotated with cost)

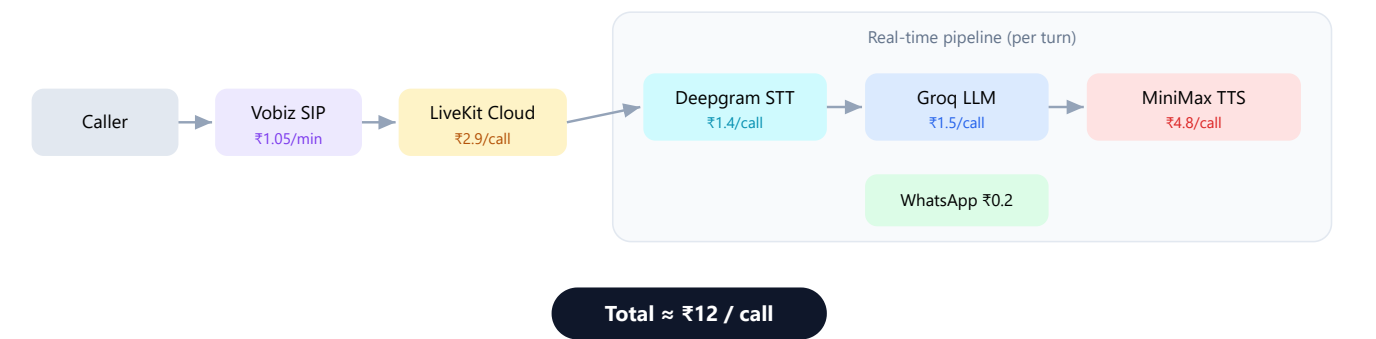


Figure 2 — Every call today goes through LiveKit + MiniMax TTS, the two most expensive links.

The problem in one sentence: every call — even a 3-line "what are your timings?" — pays full price for the premium real-time layer and freshly-synthesized speech.

4. Target architecture (two tiers)

We split traffic into a cheap **Standard** path and a premium path. Most calls take the cheap path.

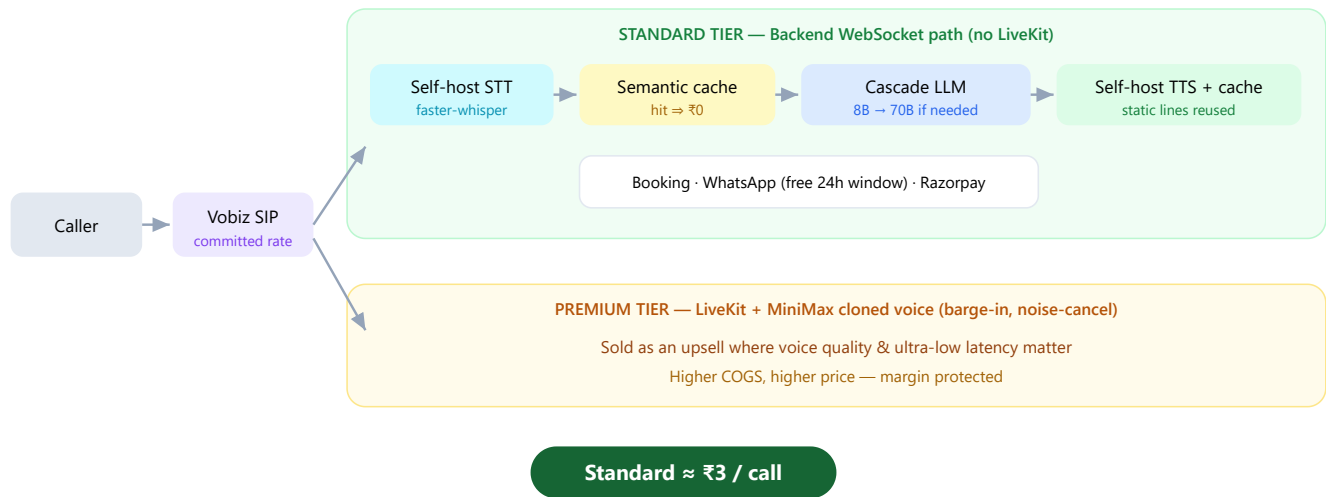


Figure 3 — Standard tier removes LiveKit, self-hosts speech, and caches everything repeatable. Premium keeps the high-end stack as a paid upsell.

5. Two key mechanisms

5a. Semantic cache + model cascade (LLM)

Most receptionist turns are repetitive ("timings?", "address?", "do you take walk-ins?"). We answer those without calling a big model — or any model.

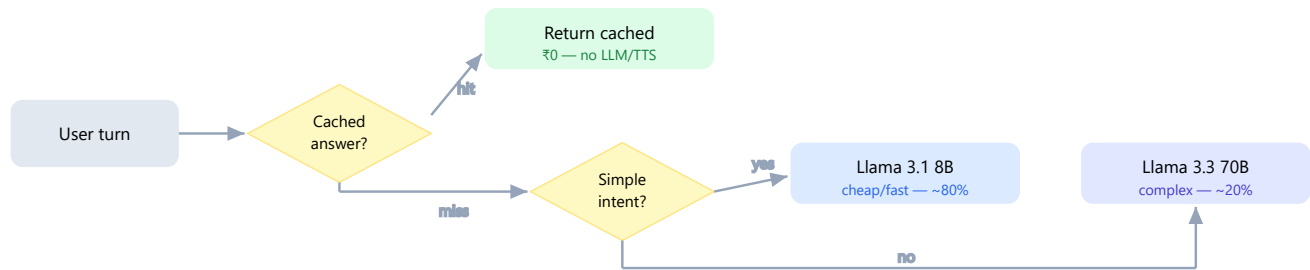


Figure 4 — Cache first, small model next, big model only when truly needed.

5b. TTS caching & self-hosting

Greetings, confirmations, token/OTP read-outs and common answers are the same every call. We synthesize them **once**, store the audio, and replay. New/dynamic speech uses a self-hosted Hindi TTS engine on the standard tier; the premium tier keeps MiniMax voice cloning.

Speech type	Today	After
Greeting, confirmations, token read-out	Synthesized every call	Pre-rendered once, cached (₹0)
Dynamic replies (standard tier)	MiniMax paid	Self-hosted GPU TTS (~₹0 marginal)
Dynamic replies (premium tier)	MiniMax	MiniMax cloned voice (kept)

6. The changes we will make

Grouped by stage. "Where" points to the actual files in our repo.

Stage 1 Quick wins — no new infrastructure (₹12 → ₹5.6/call)

#	Change	Where in code	Saves/call	Risk
1	Route standard tenants through the WebSocket path instead of the LiveKit agent	<code>backend/websocket/handler.py</code> , add tenant flag; <code>agent/main.py</code>	₹2.9	med
2	Cache static/greeting TTS lines	<code>tts.py</code> , <code>agent/minimax_tts.py</code>	₹2.0	low
3	Trim system prompt (~1,800→~700 tokens); stop re-sending KB every turn	<code>agent/main.py</code> (prompt build), <code>llm.py</code>	₹0.9	low
4	Default LLM to Groq; tighten <code>end_call</code> to cut dead air	<code>.env</code> , <code>agent/main.py</code>	₹0.6	low

Stage 2 Structural — at scale (₹5.6 → ₹3.0/call)

#	Change	Where	Saves/call	Risk
5	Self-host TTS (Piper/Kokoro/XTTS Hindi) for standard tier	New TTS worker; swap in <code>tts.py</code>	₹2.0	med
6	Semantic FAQ cache before the LLM call	New cache layer in <code>llm.py</code>	₹0.6	low
7	Model cascade (8B for simple turns, 70B on escalation)	Router in <code>llm.py</code>	₹0.4	med
8	Self-host STT (faster-whisper) past ~50k call-min/mo	New STT worker; swap in <code>stt.py</code>	₹0.8	med
9	Vobiz volume commit + WhatsApp inside free window	Commercial + <code>whatsapp.py</code>	₹0.3	low

Fixed / overhead reductions

- Self-host Postgres + Redis beyond ~5,000 tenants (vs Supabase) — saves \$500–1,500/mo.
- Autoscale agent workers to zero when idle; reserved/spot instances; host in an India region to cut egress + latency.
- Consolidate the two pipelines: WebSocket as default, LiveKit as the premium add-on (less code to maintain).

7. Phased roadmap

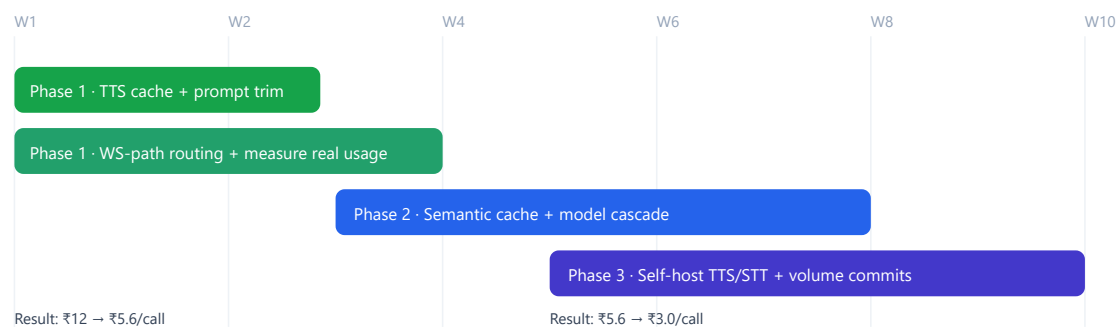


Figure 5 — Ship cheap wins in ~2 weeks; structural savings land as volume grows.

Phase	Timeline	Deliverables	Exit criteria
1 — Quick wins	Week 1–2	TTS cache, prompt trim, WS routing flag, Groq default	Cost/call $\leq$ ₹6; no quality regression on test calls
2 — Smart routing	Week 3–6	Semantic FAQ cache, 8B/70B cascade	$\geq$ 60% turns served by cache or 8B
3 — Self-host	Week 7–10	GPU TTS worker, faster-whisper STT, Vobiz commit	Cost/call $\leq$ ₹3.5 at target volume

8. Risks & trade-offs

Change	Trade-off	Mitigation
Drop LiveKit (standard)	Loses barge-in, noise-cancel, lowest latency	Keep LiveKit for a paid premium tier; A/B test quality
Self-hosted TTS	Open Hindi voices aren't as good as MiniMax cloning	Standard tier only; premium keeps cloned voice
Model cascade (8B)	8B may mishandle complex turns	Confidence threshold auto-escalates to 70B
Self-hosted STT	GPU ops burden; only pays off at volume	Switch on past ~50k call-min/mo crossover
Aggressive caching	Stale answers if business info changes	Cache keyed to tenant + KB version; invalidate on edit

9. Decisions we need from the team

- **Tiering:** Approve a two-tier model (cheap Standard vs premium LiveKit)?
- **Voice quality bar:** Is self-hosted Hindi TTS acceptable for Standard, or must all calls keep MiniMax?
- **Sequencing:** Ship Stage 1 now, or wait and do everything together? (Recommendation: ship Stage 1 now.)
- **Ownership:** Assign owners for TTS cache, WS routing, and the cascade/cache layer.
- **Measurement:** OK to instrument real calls first (actual TTS chars + tokens) so savings are exact, not estimated?

Recommended immediate next step. Approve Stage 1 and let engineering ship items 2 & 3 (TTS cache + prompt trim) this week — they are zero-tradeoff and cut cost ~25% on their own. In parallel, instrument one week of real calls so Stage 2 numbers are exact.

Assumptions: 3-min average call; ~300 answered calls/tenant/month; agent speaks ~1,000 chars/call (prompt enforces short replies); exchange rate ₹96 = \$1 (verified 27 Jul 2026). Verified provider rates: Deepgram nova-2 \$0.0043/min · MiniMax TTS ~\$0.05/1k chars · Groq Llama-3.3-70B \$0.59/\$0.79 per M tokens · LiveKit ~\$0.01/min. Vobiz has no public pricing — telephony estimated from Indian market comparables (Exotel/Plivo) and marked as an assumption. All figures are planning estimates for team discussion, not a billing guarantee.